

Sentiment Analysis Report by Wa-Hae Koela

1. Description of the Dataset

The dataset utilized for this project is the **Datafiniti Amazon Consumer Reviews of Amazon Products** (May 2019), sourced via Kaggle.

- **Primary Feature:** The primary feature variable extracted for model analysis is reviews.text, which contains unstructured text reviews submitted by Amazon consumers.
- **Data Volume:** The dataset contains over 28,000 recorded consumer entries.
- **Data Quality & Handling:** Missing values (NaN entries) within the target column were identified and dropped to prevent runtime execution errors during tokenization.

2. Details of the Preprocessing Steps

Text preprocessing was carried out using **Python**, **pandas**, and **spaCy** (en_core_web_md) to convert raw customer feedback into clean, normalized tokens suitable for NLP evaluation.

```
# Function to perform text preprocessing
def preprocess_review(text):
    # Tokenize text using spaCy medium model
    doc = nlp(text)
    # Remove stop words, punctuation, and leading/trailing whitespace
    clean_tokens = [
        token.text.lower().strip()
        for token in doc
        if not token.is_stop and not token.is_punct and token.text.strip()
    ]
    # Rejoin tokens into a normalized string
    return " ".join(clean_tokens)
```

Key Preprocessing Steps Applied:

1. **Handling Missing Values:** Applied dropna(subset=['reviews.text']) to eliminate empty reviews from the evaluation pipeline.
2. **Case Normalization:** Converted all review strings to lowercase using .lower() to standardize tokens across the corpus.
3. **Whitespace Stripping:** Utilized .strip() to clear unnecessary padding and formatting spaces.
4. **Stop-Word & Noise Filtering:** Leveraged spaCy's .is_stop attribute alongside .is_punct to strip out common filler words (e.g., "the", "is", "of") and punctuation marks, isolating

high-value sentiment words.

3. Evaluation of Results

Sentiment classification was implemented by extending the spaCy pipeline with **spacytextblob**.

Model Output & Metrics:

- **Polarity Score:** Polarity measures sentiment strength ranging from -1.0 (highly negative) to +1.0 (highly positive), where 0.0 represents neutral sentiment.
- **Sentiment Categorization:**
 - **Positive:** Polarity > 0.1
 - **Neutral:** Polarity between -0.1 and 0.1
 - **Negative:** Polarity < -0.1

Sample Tests & Verification:

Test Case	Sample Input Excerpt	Polarity Score	Assigned Sentiment
Negative Review	"item is bad quality missing backup spring..."	-0.450	Negative
Positive Review	"Works like a charm and delivery was super fast..."	+0.411	Positive
Semantic Similarity	Review A vs. Review B Comparison	0.9510 (Similarity)	High Semantic Match

4. Model Strengths and Limitations

Strengths:

- **Efficiency & Scalability:** spaCy's optimized C-extensions enable fast vector loading and processing across thousands of rows.
- **Modular Code Structure:** Wrapping data loading, text cleaning, sentiment prediction, and similarity metrics into dedicated functions ensures high readability and adherence to PEP 8 standards.

- **Linguistic Parsing:** Combining spaCy's native token filtering with TextBlob's pattern analyzer provides balanced sentiment scoring without needing complex neural network training.

Limitations:

- **Context & Sarcasm:** Rule-based polarity models struggle to detect nuanced sarcasm, double negatives, or figurative phrasing (e.g., "Yeah, right, great job breaking on day one" can yield false positives).
- **Domain-Specific Vocabulary:** General-purpose word vectors may misinterpret e-commerce technical jargon or specialized phrasing.
- **Lack of Deep Contextual Embeddings:** Unlike modern transformer models (e.g., BERT), spacytextblob evaluates words without deep dynamic bidirectional context.